MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

Project Title

IAP491_G..		
Group Members	Họ tên SV 1	Mã SV
	Họ tên SV 2	Mã SV
	Họ tên SV 3	Mã SV
	Họ tên SV 4	Mã SV
	Họ tên SV 5	Mã SV
Advisor	Học vị. Họ tên GV	
Co-advisor (if any)	Học vị. Họ tên GV	
Capstone Project code		

- 11/2025 -

©IA Dept., FPT University, Hanoi

Acknowledgments

No one achieves something great without the help of those around them, whether directly or indirectly. To complete this capstone project, we were fortunate to receive tremendous support and assistance from our supervisors, lecturers, friends, and family. We would like to dedicate these opening pages to express our sincere gratitude to everyone who has accompanied us throughout this journey.

First and foremost, we would like to express our deepest appreciation to all the lecturers at FPT University Hanoi Campus, particularly those in the Information Technology Department. The valuable knowledge they have imparted, along with their dedicated support throughout the implementation process, has enabled us to complete this capstone project and achieve noteworthy results.

We would like to extend our special gratitude to our supervisor, MSc. Trịnh Tiến Đạt, and our co-supervisor, Dr. Nguyễn Anh Nhật, who have inspired us, provided dedicated guidance, and offered invaluable support in terms of knowledge. They have created a favorable environment for us to learn and exchange ideas with fellow students in the research group. These are precious knowledge and experiences that will benefit us not only in this capstone project but also in our future careers.

Finally, we wish to express our heartfelt thanks to our families and loved ones, who have always been our solid support and have encouraged every decision we have made.

Despite our best efforts to make this thesis as complete as possible, shortcomings and limitations are inevitable. We sincerely hope to receive your understanding and constructive feedback from our supervisors and readers.

Contents

Acknowledgments ii

Contents iii

List of Figures vi

List of Tables vii

Abstract viii

1 Introduction 1

1.1 Background 1

1.2 Problem Statement 2

1.3 Research Objectives and Research Questions 3

1.3.1 Objectives 3

1.3.2 Research Questions 4

1.4 Significance of the Study 5

1.5 Scope and Limitations 6

1.6 Thesis Structure 7

2 Literature Review 9

2.1 Review of Previous Studies 9

2.1.1 Overview of Privacy-Preserving Mechanisms in Federated Learning 9

2.1.2 Data Generation 12

2.1.3 Privacy-Preserving Generative Defenses and Research Gap 15

2.2 Summary of the Literature Review 18

2.3 Contribution of Research 18

3 Methodology **19**
3.1 Research Design 19
3.1.1 Research Type and Design Overview 19
3.1.2 System Architecture and the Two Generator Routes 22
3.1.3 Threat Model, Trust Assumptions and Privacy Units 24
3.1.4 Experimental Configurations 28
3.1.5 Requirement-Driven Recommendation Rule 30
3.2 Data Collection Methods 32
3.2.1 Dataset 32
3.2.2 Label Harmonisation and Rare Classes 34
3.2.3 Schema Harmonisation and Feature Selection 35
3.2.4 Group-Based Split and Quota Sampling 35
3.2.5 Preprocessing and Encoding 37
3.2.6 Leakage and Label-Reliability Checks 37
3.3 Sampling Data Analysis Techniques 38
3.3.1 Experimental Environment and Reproducibility 38
3.4 Limitations of the Methodology 40

4 Experimental and Results **44**
4.1 Introduction 44
4.2 Presentation of Data 44
4.3 Analysis of Results 44
4.4 Interpretation of Results 44
4.5 Comparison with Literature 44
4.6 Implications of the Results 44

5 Discussion **45**
5.1 Restate the Research Problem or Objectives 45
5.2 Summarize Key Findings 45

6 Conclusion and Future Work **46**
6.1 Conclusion 46

6.2 Future Work 46

A If Any **47**

References **48**

List of Figures

3.1 Overall research workflow of PP-FedData. 22
3.2 System architecture with both generator routes. 25

List of Tables

3.1 Research questions and their experimental design 21
3.2 Comparison of the two generator routes 24
3.3 Comparison of protection mechanisms 26
3.4 Experiment matrix of PP-FedData 28
3.5 Robustness and sensitivity branches 29
3.6 Deployment scenarios (inputs only) 32
3.7 Composition of the DoS/DDoS-MQTT-IoT dataset 33
3.8 Data provenance verification 34
3.9 Final feature schema ($D = 60$) 36
3.10 Class quotas of the training, validation, and test splits 36
3.11 Leakage and label-reliability checks 38
3.12 Experimental environment specification 39

Abstract

A good abstract is a stand-alone summary of the paper, and should summarize the key components of the manuscript. Generally, the abstract should be concise and informative within 200-300 words. As an abstract is a separate section, it should be a self-containing text (no abbreviations, no references, no URLs, no undefined concepts, etc.). For research articles, abstracts should give a pertinent overview of the work. We strongly encourage authors to use the following style of structured abstracts, but without headings: (1) Background: Place the question addressed in a broad context (research background) and highlight the purpose of the study; (2) Methods: Describe briefly the main methods or treatments applied; (3) Results: Summarize the article's main findings; and (4) Conclusions: Indicate the main conclusions or interpretations.

Keywords: cybersecurity, information security, threat analysis, resilience

Chapter 1

Introduction

1.1 Background

The Internet of Things (IoT) connects physical objects, sensors, and various devices through communication networks, enabling data-driven services in domains such as healthcare, industrial automation, and smart environments. Alongside the expansion of IoT deployments, the number of connected components exposed to cybersecurity threats increases correspondingly [1]. The Message Queuing Telemetry Transport (MQTT) protocol is commonly used for lightweight publish-subscribe communication in IoT environments. However, MQTT-based systems can become targets for denial-of-service (DoS) and distributed denial-of-service (DDoS) attacks. The DoS/DDoS-MQTT-IoT dataset, collected from a physical IoT testbed, provides normal MQTT traffic alongside ten DoS attack scenarios to facilitate the study of attacks in this context [2]. Intrusion Detection Systems (IDS) monitor network activity to identify traffic or behavior exhibiting malicious signs. Machine learning and deep learning methods have been utilized and researched for this task due to their ability to learn characteristic patterns directly from network traffic. Their effectiveness depends partly on the availability and representativeness of the training data [3]. Federated Learning (FL) allows multiple clients to collaboratively train a shared model while keeping their data local; a central server distributes the global model and aggregates updates from clients across training rounds [4]. This approach is highly suitable when IoT data is distributed across multiple devices or locations, although communication and computational costs remain critical factors to consider in resource-constrained environments [5, 6]. Keeping data local also does not equate to completely eliminating privacy concerns arising from the exchange of model updates [7].

Privacy-preserving synthetic data generation is a potential solution to address the shortage of training data for rare attack classes in IoT networks while still ensuring strict security

guarantees. Rather than relying on deep neural networks that require substantial computational resources and face convergence difficulties on edge devices [8], the probabilistic graphical model paradigm (Probabilistic Graphical Models) approaches the problem by decomposing tabular data into low-dimensional marginal frequency distribution slices (low-dimensional marginals) [9]. Through the Chow-Liu dependency tree structure (Chow-Liu Tree) [10], this approach effectively captures the core correlations between traffic features and attack labels without being affected by the curse of dimensionality. When combined with Differential Privacy (DP) [9] on contingency tables and consistent graphical inference mechanisms (such as Private-PGM), the system can reconstruct a valid global distribution to generate conditional traffic data, establishing a foundation to enhance the performance of downstream intrusion detection systems (IDS) [11, 12].

1.2 Problem Statement

Building an effective IDS for IoT networks faces multiple challenges related to data privacy, the representation of attack classes, and the practical costs of the model training process. These challenges are particularly evident when training data is distributed among clients and synthetic attack samples are considered as a supplementary detection solution. The first problem relates to privacy risks during federated training. Although FL avoids the periodic transfer of raw local data, clients must still exchange model updates with the server. Previous studies have demonstrated that shared gradients can leak information about training data in certain attack scenarios [7]. Therefore, sharing model weights instead of raw data cannot be considered a comprehensive privacy protection measure. Depending on the threat model and the type of information to be protected, supplementary mechanisms may be mandatory. The second problem concerns the representation of attack classes in the training data. Class imbalance can bias the learning process towards majority classes, making it difficult to identify classes that account for a low percentage [3]. For the DoS/DDoS detection problem in MQTT, the impact of this issue depends on the dataset version, label mapping, preprocessing pipeline, and data partitioning strategy for clients. Therefore, the class distribution of the dataset after selection and preprocessing must be empirically verified rather than assumed by default. If some attack classes lack data, synthetic samples can supplement training examples. However, merely increasing the number of samples does not prove that the generated data is useful; their actual contribution must be evaluated through data quality metrics and the detection

performance of the downstream IDS model.

The third challenge pertains to the hardware constraints of edge devices, the "curse of dimensionality," and the intricate trade-off among privacy protection, data quality, and operational overhead: on the one hand, deploying traditional distributed deep generative models (such as GANs or CVAEs) requires substantial computational resources (GPU/CPU) and large RAM capacity at client nodes, which is infeasible for hardware-constrained edge IoT devices [8]. On the other hand, tabular MQTT network traffic inherently possesses a high-dimensional structure with numerous interdependent attributes. Applying Differential Privacy (DP) directly to this high-dimensional attribute space readily leads to a state-space explosion—reaching up to 10^{19} cells [11]—causing the injected differential privacy noise to completely overwhelm the original data signal and severely degrade the fidelity of the generated distributions [9].

A direct consequence is a severe tug-of-war among three factors: the degree of privacy protection (privacy budget ε), synthetic data utility for downstream IDS models, and computational/operational overhead across the IoT infrastructure. Specifically, tightening privacy guarantees (reducing the budget ε) injects heavy DP noise that destroys the core feature correlations of rare DoS/DDoS attack classes [9, 12]. Conversely, increasing the privacy budget and expanding model capacity to improve the quality of generated data causes computational costs and system runtime to surge drastically [12]. Consequently, finding a tabular data generation solution that simultaneously ensures rigorous privacy guarantees, remains computationally lightweight for edge devices, and preserves critical feature characteristics for rare attack classes remains a major, unresolved technical bottleneck.

1.3 Research Objectives and Research Questions

1.3.1 Objectives

This study proposes PP-FedData—a privacy-preserving federated data generation framework for intrusion detection systems (IDS) in IoT-MQTT networks. The research aims to achieve four primary objectives (O1–O4):

- **Objective O1 – Federated Architecture Construction:** Model each IoT Gateway device as a client within the Federated Learning network (FL Client). Each gateway

stores raw packet traffic data locally and only transmits protected statistics to the Aggregation Server under an honest-but-curious threat model.

- **Objective O2 – Marginal-based Federated Generator Development (FedDP-Marginal):** Extend marginal-based privacy-preserving data synthesis methods (such as MST and PrivBayes) to the federated learning environment: each gateway releases noisy per-class count tables across three communication rounds. Noise can be partitioned among gateways and accumulated via the Secure Aggregation mechanism (SecAgg$^+$) (corresponding to Distributed Differential Privacy using discrete Skellam noise), or fully added locally at each gateway (Local DP). Synthetic data records are subsequently sampled from a Chow-Liu dependency tree structure trained on the released marginal statistics tables.

- **Objective O3 – Deep Generative Baseline Implementation:** Implement a federated Conditional Variational Autoencoder (CVAE) model as a deep generative baseline. The CVAE model is trained using the FedAvg algorithm and protected by DP-SGD, SecAgg$^+$, or a combination of both, under identical data partitioning and evaluation protocols to enable a head-to-head empirical comparison against the FedDP-Marginal generator.

- **Objective O4 – Comprehensive Evaluation and Deployment Guidance Formulation:** Comprehensively evaluate all privacy-preserving configurations on a held-out real test set across five dimensions (utility, fidelity, privacy, overhead, and statistical rigor). From these empirical findings, establish a transparent decision rule to map practical deployment constraints (server/client trust assumptions, privacy budget ε, communication bandwidth, prioritized attack classes) into their corresponding optimal configurations.

1.3.2 Research Questions

To fulfill the four aforementioned objectives, this research focuses on addressing four core research questions:

- **RQ1:** When raw network traffic data cannot be centralized, how much IDS utility does the synthetic data generated by the federated generator retain compared to an IDS trained on centralized real data? Does augmenting training sets with synthetic data

improve detection performance when real data is already available (either centrally or at individual gateways)?

- **RQ2:** Under the same privacy budget (ε, δ), how does the federated marginal-based generator (FedDP-Marginal) compare to the federated CVAE model trained with DP-SGD regarding downstream utility, detection capabilities on rare attack classes, and network communication overhead?
- **RQ3:** What overhead does each protection mechanism incur under DP, SecAgg$^+$, or their combination, and at what granularity does it provide privacy protection? How do the unit of privacy protection (packet, TCP stream, or capture session) and the number of honest gateways alter privacy guarantees and operational costs?
- **RQ4:** Which protection configuration (combining DP, SecAgg$^+$, noise distribution, and privacy protection unit) should be selected for specific real-world deployment requirements?

1.4 Significance of the Study

This study addresses two fundamental bottlenecks in collaborative intrusion detection across IoT-MQTT networks: (1) rare attack classes are sparsely and non-identically distributed across IoT Gateway devices (Non-IID), and (2) gateways cannot share raw packet traffic data due to privacy concerns.

The study offers three primary scientific and practical contributions:

- **Marginal-Based Distributed Generator (FedDP-Marginal):** Proposes a distributed synthetic data generation solution in which marginal statistics released by individual gateways are simultaneously protected by formal Differential Privacy (DP) and Secure Aggregation (SecAgg$^+$).
- **Controlled Benchmark:** Conducts a direct, head-to-head empirical comparison between the marginal-based generator (FedDP-Marginal) and the deep generative baseline (Federated CVAE) under identical data partitioning schemes, consistent privacy-preserving mechanisms, and a rigorous evaluation protocol.
- **Requirement-Driven Configuration Guide:** Proposes a transparent decision guideline to assist IoT system administrators in selecting the most suitable privacy configuration

based on their specific threat model, mutual trust assumptions, and practical resource budgets.

Explicitly Stated Limitations and Privacy Guarantees:

The privacy guarantees in this study are precisely and transparently defined:

- **Differential Privacy (DP)** provides mathematical upper bounds limiting the amount of information that released statistical tables or models can leak regarding a single data record (by default at the packet level, or optionally at the TCP stream or capture session level under specific assumptions). The DP mechanism does not anonymize class labels and does not enforce privacy guarantees on the hyperparameter tuning process conducted on validation data.
- **Secure Aggregation (SecAgg$^+$)** only guarantees the confidentiality of individual gateway updates/statistics with respect to the Aggregation Server; it does not prevent what the final model or generated synthetic data might leak about the aggregate dataset.

1.5 Scope and Limitations

This research is scoped within the following specific experimental and technical boundaries:

- **Dataset and Data Granularity:** Utilizes the DoS/DDoS-MQTT-IoT dataset at the packet level (where each record corresponds to a single network packet and includes six traffic class labels: one Normal class and five DoS/DDoS attack families).
- **Federated Simulation Architecture:** A network of five simulated IoT Gateway devices is deployed under an uneven data distribution across nodes (Dirichlet non-IID partitioning).
- **Generative Models and Privacy Mechanisms:** Evaluates and compares two generators: the primary marginal-based federated generator (FedDP-Marginal) and the deep generative baseline (Federated CVAE). Enforces Differential Privacy (DP) across privacy budget levels $\varepsilon \in \{1, 5, 10\}$ with $\delta = 10^{-5}$, Secure Aggregation (SecAgg$^+$), as well as a hybrid configuration combining both mechanisms.

- **Experimental Infrastructure:** The entire federated training workflow is simulated on a single physical host machine using the Flower framework.

Despite achieving rigorous empirical findings, this study is subject to several scientific limitations:

- **Single-Dataset Reliance:** All empirical evaluation results and configuration selection guidelines are derived exclusively from the DoS/DDoS-MQTT-IoT dataset; therefore, generalizability to other IoT application protocols (such as CoAP or HTTP/2) requires further empirical validation.
- **Packet-Level Independence Assumption:** Packets are modeled independently as tabular records, which does not fully capture the temporal/sequential dependencies inherent in an end-to-end communication session or stream.
- **Default Record-Level Privacy Scope:** The Differential Privacy (DP) guarantee applies by default at the individual record level (a single packet). Extending protection to an entire TCP stream or a full packet capture session represents an advanced configuration option that necessitates a trade-off resulting in degraded synthetic data utility.
- **Privacy Budget ε Exclusions:** The privacy budget ε covers only the released count statistics and trained models. Auxiliary metadata such as attack class labels, global preprocessing statistics, and validation-based hyperparameter tuning fall outside the theoretical guarantee of ε.
- **Simulation Environment:** Conducting the simulation on a centralized physical machine does not fully capture real-world network latency, packet loss rates, or the physical hardware constraints of resource-limited edge IoT devices.
- **Constrained Threat Model:** The study operates under an honest-but-curious threat model across all gateways. Sophisticated adversarial threat scenarios such as data/model poisoning attacks or Byzantine clients are outside the scope of this investigation.

1.6 Thesis Structure

The remainder of this thesis is structured into six chapters as follows:

- **Chapter 1: Introduction:** Outlines the background, problem, objectives, significance, scope, and structure of the study.
- **Chapter 2: Literature Review:** Reviews relevant studies and identifies the research gap and the position of this study in the existing literature.
- **Chapter 3: Methodology:** Details the research design, the dataset and its preparation, the two generators, the privacy mechanisms and accounting, and the evaluation protocol.
- **Chapter 4: Experimental and Results:** Describes the experimental setup and reports synthetic-data, IDS, and system overhead results.
- **Chapter 5: Discussion:** Interprets the findings, examines the observed trade-offs, and discusses the limitations and implications of the study.
- **Chapter 6: Conclusion and Future Work:** Summarizes the conclusions and outlines directions for further research.

Chapter 2

Literature Review

Reviews relevant existing research and demonstrates how your thesis fits into the broader academic conversation.

2.1 Review of Previous Studies

2.1.1 Overview of Privacy-Preserving Mechanisms in Federated Learning

Overall Landscape and Scope of Research

The integration of Federated Learning (FL) into Intrusion Detection Systems (IDS) for IoT environments has resolved the issue of raw data centralization, yet the model weight sharing process itself still harbors data leakage risks [13]. To counter this, current studies classify Privacy-Preserving Federated Learning (PPFL) techniques into five main categories: Differential Privacy (DP), Homomorphic Encryption (HE), Secure Multi-Party Computation (SMPC), Trusted Execution Environments (TEE), and Blockchain [13]. Although TEEs provide a secure execution environment and Blockchain ensures transparency, they require specialized hardware or complex consensus mechanisms, which are unsuitable for the constrained resources of edge/IoT devices [14]. Therefore, this study implements DP and SMPC-based secure aggregation (SecAgg$^+$); HE is reviewed as an alternative but not implemented, because of its computational cost on constrained devices [15].

Trusted Execution Environment (TEE)

The Trusted Execution Environment (TEE) provides a completely isolated and secure hardware computing area for processing sensitive data, effectively preserving privacy.

However, this technology is heavily dependent on specialized hardware security mechanisms and verifiable components. This strict hardware requirement makes TEE cumbersome and unsuitable for deployment across lightweight IoT networks comprising massively resource-constrained devices [13].

Blockchain

Blockchain preserves data privacy through decentralized ledgers and cryptographic algorithms that ensure tamper-resistance and eliminate the need for intermediaries. However, this system suffers from fatal performance weaknesses due to low throughput, high latency, and expensive verification costs stemming from continuously growing immutable storage. These severe latency and massive storage overheads make blockchain highly impractical for lightweight, multi-device IoT environments that demand rapid response times [13].

Homomorphic Encryption (HE)

HE enables computations to be performed directly on encrypted data, ensuring absolute security for model updates from clients before server aggregation without the need for decryption [15]. However, the computational cost and communication requirements of HE algorithms are extremely immense and heavy, leading to a surge in system latency [15]. This renders HE less feasible for systems requiring real-time responses and operating under resource constraints, such as IoT-based IDS [15].

Differential Privacy (DP)

Unlike the cumbersomeness of HE, Differential Privacy (DP) offers excellent scalability. The core principle of DP is to mask the contribution of any single data point by adding random noise to the model update at each client [16]. DP is exceptionally lightweight, consumes no extra bandwidth, and easily scales to tens of thousands of participating clients [16]. The tradeoff of this method is that injecting noise into the model inevitably degrades data accuracy, forcing engineers to constantly seek an optimal balance for the privacy budget [16].

Secure Multi-Party Computation (SMPC/SecAgg)

Conversely, Secure Aggregation (SecAgg) protocols based on SMPC comprehensively resolve the weakness of DP. SMPC prevents the server from directly observing individual client updates by ensuring that the server only receives the aggregated model update [17]. Secure aggregation adds no statistical noise; with the fixed-point quantisation used by SecAgg$^+$, the aggregate differs from the plain sum only by a bounded quantisation error [18]. As a trade-off, the original Secure Aggregation protocol requires multiple rounds of complex interactions between clients to exchange secret shares, causing severe bandwidth bottlenecks as the client population grows [19]; however, the subsequent SecAgg$^+$ protocol successfully resolves these overhead issues by replacing the complete communication graph with a k-regular graph, thereby reducing the per-client communication and computation complexity to a polylogarithmic scale [18].

Hybrid Privacy-Preserving Systems

The study by Gutti et al. [20] (integrating DP and SecAgg via the HADA algorithm) demonstrates that combining DP (to prevent inference attacks) and SecAgg (to protect the transmission channel) is a promising trajectory. However, stacking these security layers generates immense complexity in computation and hyperparameter tuning [20]. Therefore, before advancing to hybrid architectures, independently measuring the performance trade-offs between the DP pipeline and the SecAgg pipeline serves as a mandatory stepping stone to evaluate the endurance of the network system.

Distributed Differential Privacy

Distributed DP combines the two families: each client adds only a share of the noise, and secure aggregation reveals only the sum, which carries the full calibrated noise [21, 22]. Because secure aggregation operates on integers modulo a large number, discrete noise is preferred; the sum of independent Skellam variables is again Skellam, which permits exact privacy accounting after modular aggregation [22]. The guarantee assumes that enough clients add their share honestly; with fewer honest clients the effective noise is smaller. Distributed DP fits releases that are a single sum, such as count tables, but not multi-step local DP-SGD, where each client's noise passes through non-linear local training before aggregation.

2.1.2 Data Generation

Overview of Data Generation Techniques

In the field of cybersecurity and Intrusion Detection Systems (IDS) for Internet of Things (IoT) environments, Generative AI (GenAI) techniques for data generation are defined as deep generative paradigms capable of learning the underlying multivariate probability distribution of network traffic, thereby automatically synthesizing novel data samples whose structural attributes and statistical properties closely match genuine traffic distributions [23].

Core problems addressed by data generation techniques:

- **Overcoming Extreme Class Imbalance:** Real-world IoT network traffic inherently exhibits extreme class skewness, where over 90–99% of collected traffic consists of benign activity, while malicious behaviors—especially Low-Rate Stealth Attacks (LRSA) or rare DoS/DDoS variants—represent a negligible fraction. This acute shortage of attack samples severely biases downstream IDS classifiers toward majority classes, resulting in unacceptable false-negative rates on critical threats [23].

- **Beyond Interpolation-Based Oversampling:** Traditional statistical oversampling algorithms such as SMOTE [24] operate by performing linear interpolation between existing minority instances within a localized convex hull. As highlighted by Deng et al. [23], deep generative models surpass these rigid boundaries by capturing the global multivariate distribution, enabling the synthesis of samples in unexplored regions of the feature space to cover edge cases and zero-day attack variants, thereby strengthening model robustness. Furthermore, because SMOTE requires simultaneous access to neighboring minority instances in a unified feature space, it is fundamentally ill-suited for federated environments where an individual edge gateway may hold few or no records of a rare attack class [24].

- **Supporting Data Augmentation and Restructuring:** Systematic survey statistics indicate that up to 89% of studies leverage GenAI specifically for data augmentation and class rebalancing, substantially improving the detection performance and generalization of IDS without exposing sensitive raw telemetry [23].

Primary taxonomies of generative architectures in IDS:

- **Generative Adversarial Networks (GANs):** Architectures structured around an adversarial minimax game between a Generator network that creates synthetic samples

from latent noise and a Discriminator network that evaluates sample authenticity. Due to their capacity to model sharp, high-dimensional distributions, GANs account for an overwhelming 58% of research in generative IoT-IDS [23].

- **Autoencoders and Variational Autoencoders (AEs/VAEs):** Frameworks comprising an Encoder that maps high-dimensional traffic into a low-dimensional latent manifold and a Decoder that reconstructs the input. VAEs enforce continuous probability distribution priors over the latent space, permitting stable random sampling to synthesize realistic minority samples [23].

- **Transformer-Based Generative Models:** Architectures leveraging multi-head self-attention mechanisms to capture complex contextual correlations and sequential packet dependencies across temporal network traffic flows [23].

Analysis of Strengths and Weaknesses of Techniques in Existing Studies

Although data generation techniques provide substantial potential for IoT-IDS, critical examination of existing literature reveals distinct computational bottlenecks, training instabilities, and privacy trade-offs when deployed in decentralized Federated Learning (FL) settings [23]:

- **Heavy Generative Denoising Techniques (Tabular Diffusion / Tab-DDPM):** Manzano et al. [14] integrated tabular diffusion models (Tab-DDPM) directly at local FL clients to synthesize minority samples on the BoT-IoT benchmark. While Tab-DDPM achieves exceptional sample fidelity and distribution alignment, the reverse diffusion process requires iterative denoising over hundreds of sequential steps. On edge-tier testbeds (1 CPU core, 2 GB RAM), generating a single synthetic sample requires approximately 32 seconds [14]. Such extreme computational latency is completely unviable for line-rate packet inspection in edge IoT environments.

- **Improved Policy-Guided GANs:** Yang et al. [25] employed an improved GAN guided by reinforcement learning to augment training sets for DDoS detection. While ablation experiments confirm that removing synthetic augmentation causes a noticeable drop in detection performance, the multi-component architecture requires up to 2.726 seconds per batch and consumes 19.6 GB of GPU memory on the KDDcup99 dataset [25].

Furthermore, the study did not isolate standalone generator inference latency, obscuring the true computational efficiency of the generator on edge hardware.

- **Survey Insights on Pure GAN Instability:** In their survey, Deng et al. [23] highlight that while GANs produce high-fidelity samples, the adversarial training dynamics are notoriously brittle and susceptible to mode collapse—wherein the generator repeatedly outputs a narrow set of duplicate patterns, wasting node compute without contributing meaningful diversity.

- **Random Resampling Techniques (Bootstrapping / Oversampling):** Jacob and Sultana [26] utilized bootstrapping (random resampling with replacement) at local nodes to balance the BoT-IoT and CSE-CIC-IDS2018 datasets. While computationally trivial and easily executable on microcontrollers, bootstrapping merely duplicates existing data points without expanding feature diversity. This encourages classifier overfitting and fails to generalize against evolving attack variants or concept drift [23, 26].

- **Cloud-Based Centralized Data Generation Techniques (Cloud-Based VAE-GAN):** Anandhi and Srikanth [27] proposed the SSAIM framework utilizing a hybrid VAE-GAN at the cloud coordinator to generate 51,200 Low-Rate Stealth Attack (LRSA) samples for the IoTID20 dataset, raising attack recall from 62% to 98%. However, placing data synthesis in the cloud violates the decentralized privacy principle of FL. Additionally, compressing the heavy VAE-GAN model via fixed-point quantization for transmission back to edge devices demonstrated that quantization below 6-bit (e.g., 4-bit) caused a severe degradation in detection sensitivity [27].

- **Federated Lightweight Edge GANs (TinyGAN in FL):** Durga et al. [28] deployed TinyGAN within a federated framework for IoMT malware detection, reporting 99.30% accuracy after 20 rounds. However, to stabilize GAN training, all malware classes representing less than 0.5% of the dataset were discarded, thereby sidestepping the core challenge of extreme class imbalance. Moreover, the study omitted standalone sample generation throughput measurements, leaving its actual computational advantage on constrained edge hardware unproven [28].

2.1.3 Privacy-Preserving Generative Defenses and Research Gap

Privacy-Utility Dilemma in Federated Generative IDS

Federated intrusion detection systems (IDS) on edge networks must protect client telemetry while coping with severe minority-attack-class imbalance and non-IID traffic distributions. Differential privacy (DP) and cryptographic protocols bound gradient inversion and membership inference risks [29] but do not correct class skew. Augmenting under-represented attack classes with federated generative models mitigates this imbalance, yet generative architectures may memorize training records and expose sensitive network traces through parameter exchanges or synthesized flows [29]. Edge defenses must therefore limit the utility loss of synthetic data while providing formal client-side privacy guarantees.

Existing privacy-preserving generative IDS frameworks use cryptographic primitives, DP, or both. Cryptographic mechanisms, such as secure multi-party computation (MPC) and threshold secret sharing, preserve numerical precision without perturbation error [29]. However, secure aggregation of high-dimensional neural parameters requires iterative multi-round communication, which burdens resource-constrained IoT edge devices [29].

Empirical Generative PPFL-IDS and Cryptographic Limitations

A literature search of IEEE Xplore and SpringerLink (2022–2026) for studies combining federated learning, intrusion detection, and generative models (GAN or VAE) identified five empirical implementations, all from IEEE [30–34]. None applies cryptographic secure aggregation to uplink generator updates. Salim et al. [32] (global VAE aggregation), Jiao et al. [33] (local CGAN gradients), and Shettar et al. [34] (central aggregator with moments-accountant clipping at $\varepsilon = 4.8$ and $\delta = 10^{-5}$) rely on DP alone. He et al. [31] combine local gradient noise with blockchain Proof-of-Authority consensus across edge servers, which ensures aggregation integrity and auditability rather than parameter confidentiality.

Yao et al. [30] use a hybrid design: proactive dynamic (t,n)-threshold secret sharing with $t = 4$ and $n = 5$ protects only downlink synthetic broadcasts, whereas uplink client gradients rely on Gaussian DP. They report execution measurements for key distribution and sample reconstruction (Table VI), but secret sharing covers downlink tabular samples, not full model-parameter aggregation. Outside the primary search, Hamouda et al. [35] likewise omit secure aggregation and identify cryptographic model protection as an open challenge. Because

none of these works benchmarks cryptographic aggregation against DP on model weights, their use of DP reflects a design choice rather than a measured trade-off.

The effect of DP perturbation on downstream detection depends on the privacy budget ε, gradient clipping threshold C, and minority-class sample density, yet existing studies do not isolate these factors. Utility can remain high at a moderate budget or with large synthetic volumes: Shettar et al. [34] report a 96.8% detection F1-score under central DP ($\varepsilon = 4.8$), and Hamouda et al. [35] observe a 0.25-percentage-point accuracy reduction at $\varepsilon = 1.0$ when applying DP-SGD to the downstream global classifier with 50,000 synthetic samples per class. Generative augmentation can also raise rare-class detection from sparse baselines: He et al. [31] report that CGAN augmentation improved Infiltration F1-score from 33.33% to as high as 98.01%, starting from 29 baseline training samples. However, Hamouda et al. [35] applied DP to the classifier rather than the generator, and neither He et al. [31] nor Jiao et al. [33] report formal numerical (ε, δ) budgets. Because generator-side DP must spread privacy noise across iterative training steps, how strict bounds ($\varepsilon \leq 1.0$) interact with extreme class scarcity in federated generative architectures remains an open empirical question.

Marginal-Based Tabular Synthesis vs. Deep Generative Models

Marginal-based tabular synthesis offers an alternative to deep generative models. Instead of optimizing non-convex neural network weights by backpropagation, marginal synthesizers select and measure low-order contingency tables and reconstruct the data distribution with probabilistic graphical models. Beyond network IDS research, a cross-domain search of the ACM Digital Library, PMLR, and arXiv identified distributed extensions: Maddock et al. [36] introduce FLAIM, which adapts the Adaptive and Iterative Mechanism (AIM) to distributed clients under zero-concentrated differential privacy (ρ-zCDP) and compares DP with secure multi-party computation (DistAIM); Pentyala et al. [37] present CaPS, a "DP-in-MPC" framework in which secret sharing protects marginal selection and measurement; and Li et al. [38] propose HeteroFedSyn, which compresses two-way marginal dependencies by random projection under distributed perturbation.

Marginal synthesizers avoid the adversarial minimax training instability of GANs but carry their own privacy-utility trade-offs. Iterative selection-and-measurement algorithms (e.g., AIM and Private-PGM) divide the total privacy budget ε across rounds, and distributing measurements across federated clients compounds perturbation noise [36, 37]. In sparse discrete categories with small contingency cell counts, additive Laplace or Gaussian noise

yields a low signal-to-noise ratio. Restricting measurements to low-order marginals (typically two-way dependencies [38]) truncates higher-order multivariate correlations, and continuous flow variables require discretization, which introduces binning artifacts.

Federated marginal generators have so far been evaluated mainly on general-purpose tabular benchmarks from demographic, financial, and biomedical domains (e.g., Adult, Census, Credit Card Fraud, COMPAS, Diabetes) [36–38]. FLAIM [36] also used the legacy DARPA Intrusion Detection benchmark but evaluated only workload error and negative log-likelihood, not downstream minority-class intrusion classification. Decentralized tabular GANs with cryptographic primitives also exist: Duan et al. [39] combine Paillier additively homomorphic encryption for global multimodal distribution encoding with DP during GAN training, but evaluate statistical utility and membership inference resistance on legacy tabular data (KDD Cup 99), not modern protocol telemetry.

IoT-MQTT Protocol Telemetry and Working Hypotheses

IoT-MQTT telemetry combines structured, low-cardinality discrete protocol fields (e.g., message types, QoS levels, retain flags) with continuous flow metrics (e.g., payload lengths, burst durations). Sparse discrete feature combinations form low-count contingency cells, where both generative families risk degradation under DP noise, making this a suitable setting for examining these trade-offs. We hypothesize that, under generator-side DP at strict budgets ($\varepsilon \leq 1.0$), a federated marginal synthesizer achieves higher minority-class detection F1-score than a federated deep generative model for attack classes defined by low-order (two-way) discrete header correlations (e.g., MQTT Unauthorized Connect, Malformed Header). We expect the reverse for volumetric or timing-based attacks (e.g., MQTT Publish Floods) governed by multi-attribute continuous flow dynamics. We test this across five random seeds, equalizing privacy budgets by converting all mechanisms to (ε, δ)-DP under Rényi differential privacy accounting, and compare class-specific F1-scores with a Wilcoxon signed-rank test (treating a margin $|\Delta\text{F1}| \geq 2.0$ percentage points at $p < 0.05$ as statistically significant).

Identified Research Gaps

To the best of our knowledge, the IEEE, Springer, ACM, PMLR, and arXiv searches described above indicate three distinct research gaps:

- **Marginal Synthesis for Protocol Telemetry:** Deep generative models have been evaluated on MQTT-bearing datasets (e.g., Hamouda et al. [35], federated conditional GAN on Edge-IIoTset), but no prior study evaluates a federated marginal-based generator on modern IoT-MQTT telemetry or examines how mixed discrete-continuous features affect marginal synthesis relative to deep models.
- **Controlled Architectural Comparison:** No empirical study compares a federated marginal-based synthesizer with a federated deep generative model (e.g., CGAN, VAE) under identical non-IID client partitions, minority-attack imbalance ratios, and calibrated generator-side privacy budgets (ε, δ).
- **Downstream Detection Trade-offs:** Prior marginal studies compared DP with cryptographic aggregation on workload error, negative log-likelihood, runtime, and communication bandwidth [36, 37], and decentralized GANs were evaluated for statistical utility and membership inference resistance [39]. No study quantifies how choosing between DP and cryptographic secure aggregation affects downstream minority-class intrusion detection under matched federated edge conditions.

This study addresses these gaps by evaluating communication overhead, synthetic sample fidelity, and downstream minority-class intrusion detection for marginal and deep generative models under both DP and cryptographic secure aggregation on edge IoT networks.

2.2 Summary of the Literature Review

2.3 Contribution of Research

Chapter 3

Methodology

Describes the research design, data collection methods, and analysis techniques.

3.1 Research Design

3.1.1 Research Type and Design Overview

This study is a quantitative, controlled experimental study carried out on one published network-traffic dataset, DoS/DDoS-MQTT-IoT [2]. In the terminology of the FPT thesis guide, it combines Experiments with Archival Research: the independent variables (generator, privacy mechanism, privacy budget) are manipulated by the researchers, while the traffic itself is existing recorded data rather than data collected specifically for this study.

Research Object

The object of study is a framework, PP-FedData, that lets several IoT/MQTT gateways jointly produce synthetic network-traffic records without any gateway releasing its raw packets, so that an intrusion detection system (IDS) can be trained on data that no single party could pool. The framework is evaluated along four dimensions: the utility of the synthetic data for the IDS, the privacy guarantee attached to the released artefacts, the communication and computation overhead, and the fit of each configuration to a stated deployment requirement.

Experimental Control

Internal validity rests on holding everything except the studied factors fixed. Every configuration in the experiment matrix (Section 3.1.4) is run with:

- the same train/validation/test split, formed by capture group, with a real test split that is locked and never used for tuning (train 88,500, validation 12,000, test 30,000 rows; class quotas fixed in the configuration);
- the same non-IID partition of the training rows over $K = 5$ clients, drawn from a Dirichlet distribution with concentration $\alpha = 0.5$ [40] and fixed by the seed;
- the same preprocessing ($\log(1+x)$ transformation of skewed columns, standardisation, clipping at $\pm 5\sigma$, one-hot encoding of categorical columns), which turns the harmonised packet attributes into 60 model input columns;
- the same downstream classifiers: a Random Forest [41] with 200 trees and a multilayer perceptron with hidden layers $[128, 64]$;
- the same three random seeds $\{0, 1, 2\}$;
- the same real test split for every reported score.

Under this design, differences between configurations are attributable to the generator and to the protection mechanism, subject to the limits listed in Section 3.4. The design does not claim that extraneous influences are eliminated: hyper-parameter selection, the choice of capture groups, and the number of seeds all remain sources of variation, and Section 3.4 states how far each is covered.

Research Questions and Design Mapping

Table 3.1 maps each research question to the comparison, the evaluation protocol, and the measures used to answer it. The protocols are defined in Section 3.3.6. Training-and-testing notation follows common usage: TRTR trains and tests on real data; TSTR trains on synthetic data and tests on real data; TAug trains on real plus synthetic data; TAugR adds synthetic rows only for the rare classes; local augmentation supplements each client's own classifier with synthetic rows for classes it lacks.

The rare classes are DELAYED, SYN, INVALID, and WILL, the four attack families whose training quota is at most 3,000 rows. The macro-F1 is the unweighted mean of the per-class F1 over the six classes; binary F1 separates NORMAL from the merged attack classes.

Table 3.1: Research questions and their experimental design

RQ	Question (short form)	Compared configurations	Protocols	Primary measures
RQ1	Does synthetic data improve an IDS relative to limited real data?	B0, B1a, B1b (pooled real references) against B3, M2 and MG (synthetic)	TRTR, TSTR, TAug, TAugR, local augmentation	Macro-F1 over six classes; mean recall of the rare classes
RQ2	How do the marginal-based and deep generator routes compare under the same DP budget?	M1, M1o, M3, M3o (federated CVAE with DP) against MG, at $\varepsilon \in \{1, 5, 10\}$	TSTR	Macro-F1; rare-class recall; MB per round; number of rounds
RQ3	What does privacy cost in utility and in resources?	B3 against M1 (cost of DP); B3 against M2 (cost of SecAgg); MGs against MGl; suffixes `-strm`, `-cap`, `-t`	TSTR; privacy measurements; cost monitoring	ε as a function of honest clients h; membership-inference AUC; bytes; time
RQ4	Which configuration suits which deployment requirement?	All valid rows of the scorecard	Requirement-driven rule (Section 3.1.5)	Satisfaction of the requirement vector

Overall Workflow

The research proceeds in five stages (Figure 3.1): data preparation with leakage checks; partitioning over clients; training of the generator routes; multi-protocol evaluation; and aggregation into a scorecard from which recommendations are drawn.

Evolution of the Design

The study was originally designed around a federated conditional variational autoencoder (CVAE) [42], which is the premise of the project. After a first evaluation round, the framework adopted a marginal-based generator, FedDP-Marginal, as its primary generator and retained

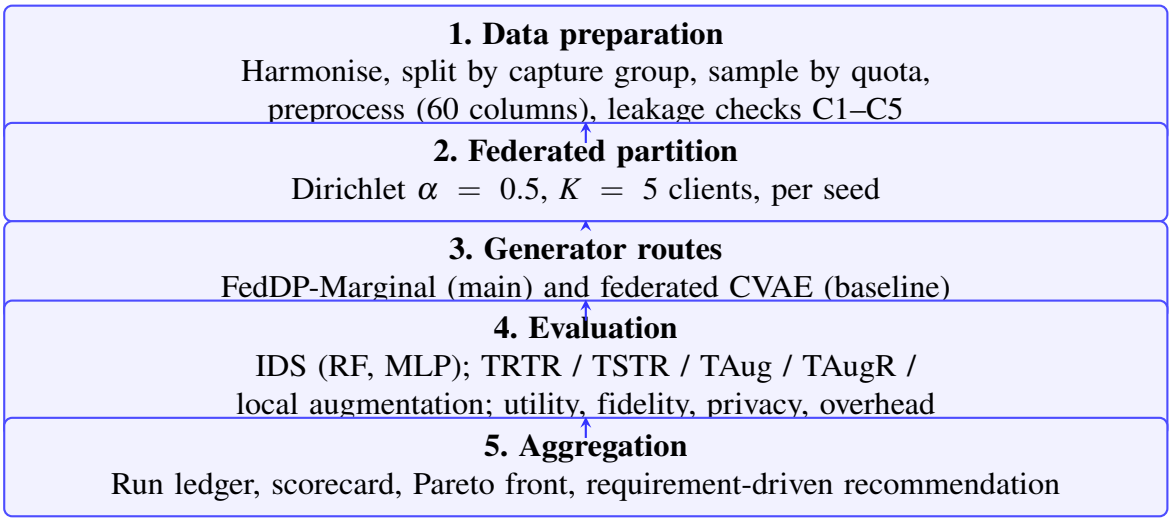


Figure 3.1: Overall research workflow of PP-FedData.

the CVAE as a baseline. The decision is recorded with its date and rationale in the project's deviation log (`docs/SPEC_DEVIATIONS.md`, entries O1.9 and O2.3); Chapter 4 presents the evidence and Chapter 5 discusses it. The CVAE configurations remain in the matrix so that the comparison in RQ2 is made under identical federation, privacy, and secure-aggregation conditions.

3.1.2 System Architecture and the Two Generator Routes

Roles

The system involves three primary roles (Figure 3.2):

- **Gateways (federated learning clients):** The main study uses $K = 5$ gateways. Each gateway holds the traffic it observed and is trusted with its own data. Because the partition is non-IID, some gateways have few or no examples of some attack classes.
- **Aggregation server:** The server coordinates the rounds, combines client contributions and, after training or table release, produces the synthetic data. It is modelled as honest-but-curious (Section 3.1.3).
- **Downstream IDS:** A Random Forest and a multilayer perceptron are trained on synthetic data (TSTR) or on real plus synthetic data (TAug, TAugR, local augmentation) and are evaluated on the real test split.

Where Synthetic Data is Generated

Synthetic records are generated at the server, from the global model (CVAE decoder) or from the released global tables (marginal counts and dependency tree), by sampling conditionally on the class label. They are not generated at the gateways. The sampling functions are implemented in `models/generate.py` (CVAE) and `models/marginal.py` (FedDP-Marginal).

What the Server Sees

The visibility of client contributions depends on the configuration and must not be stated in general terms:

- M2, M3, MGs, and MGr use Flower's SecAgg$^+$ protocol [18, 43]: the server receives masked vectors and can recover only their sum.
- B3 and M1 use plain FedAvg [4]: the server sees each client's model update; in M1 the update already carries noise added by DP-SGD.
- MGl (local DP): each client adds the full noise to its own tables and sends them without secure aggregation, so the server sees each client's noisy tables.
- MGd and MGb simulate the distributed sum inside one process; they do not pass through the SecAgg$^+$ protocol.

The Two Generator Routes

Table 3.2 contrasts the two routes. FedDP-Marginal exchanges counts of discrete attribute combinations in three releases; the CVAE exchanges continuous model weights over many FedAvg rounds. The difference matters for privacy accounting. Each marginal release is a single additive aggregate with bounded sensitivity, so the calibrated noise can be divided among the clients and the sum computed through secure aggregation. DP-SGD [44], by contrast, performs many local non-linear steps per round, and noise added by one client passes through the other clients' training before aggregation; the project therefore does not claim distributed DP for the CVAE route (`SPEC_DEVIATIONS.md`, O2.1).

For the FedDP-Marginal route, the sum is carried by a patched client path: stock Flower SecAgg$^+$ sends masked vectors as 64-bit integers, so the project casts the masked vector to

Table 3.2: Comparison of the two generator routes

Property	FedDP-Marginal (main generator)	Federated CVAE (baseline)
Quantity shared	Count tables (summed over clients) in three releases: class-conditional 1-way marginals, pairwise tables, tables for dependency tree edges	Model weights, 30 FedAvg rounds with 2 local epochs (default configuration)
Model structure	Chow–Liu dependency tree [10]; optional second-order Bayesian network (PrivBayes-style [9], MGb)	Conditional VAE [42] with an encoder and a block-wise decoder
Where DP noise is added	On each client's tables, once per release (Skellam [22] or Gaussian share)	On each client's per-example gradients at every local step (DP-SGD via Opacus [45])
Secure aggregation	Flower SecAgg$^+$ on exact integers (uint32, arithmetic modulo 2^{32})	Flower SecAgg$^+$ on fixed-point quantised weights (2^{22} quantisation levels, clipping range 16)
Distributed DP	Valid: each release is one additive aggregate	Not claimed: multiple non-linear local steps
Main code	`models/marginal.py`, `fl/mg_app.py`, `group_dp.py`	`models/cvae.py`, `fl/core.py`, `fl/app.py`, `fl/dp_utils.py`

uint32 after the final modular reduction and lets the server's unsigned addition wrap, which equals addition modulo 2^{32} (`SPEC_DEVIATIONS.md`, O2.2). The installed Flower package is not modified; the patch lives in the project process.

3.1.3 Threat Model, Trust Assumptions and Privacy Units

Adversary and Trust Assumptions

The aggregation server is assumed to be honest-but-curious, also called semi-honest [46]: it follows the protocol but may analyse every message it receives. Gateways hold their own traffic and are trusted with it. The study considers four trust settings, which differ in how much the privacy guarantee depends on other gateways:

- **Base setting:** All K gateways follow the protocol and each adds its designated share of the noise ($1/K$ of the variance in the distributed variants).

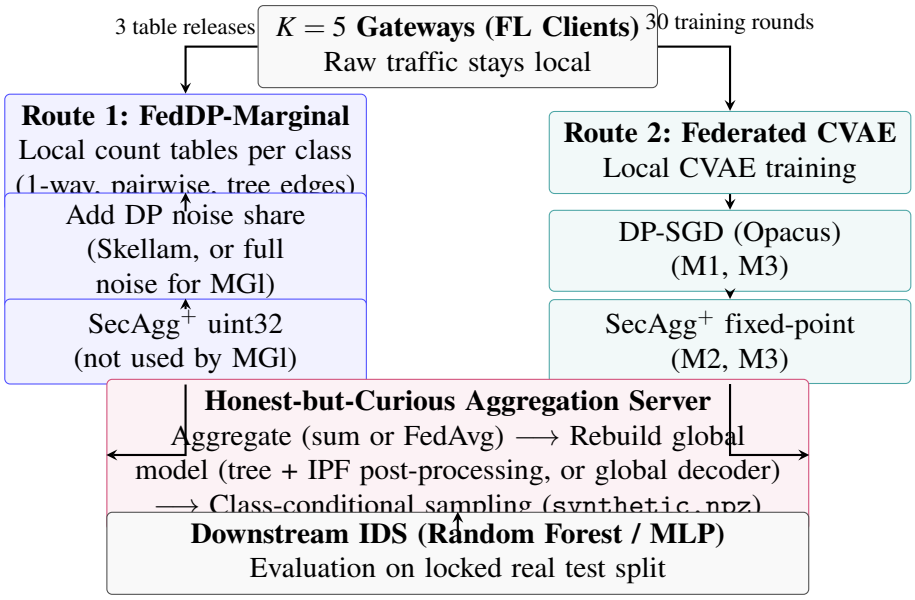


Figure 3.2: System architecture with both generator routes.

- h **honest gateways:** The guarantee is also reported when only $h \in \{1,\ldots,K\}$ gateways add their share; the remaining gateways may collude with the server or drop out. The reported ε then grows as h decreases. For the distributed FedDP-Marginal variant, the project reports ε for every h; Chapter 4 gives the values.
- **Threshold** t**:** The per-client noise is calibrated so that the target (ε, δ) holds whenever at least t gateways contribute their share. Each client then adds $1/t$ of the calibrated variance. The default is $t = K$.
- **Local DP (MGl):** Each gateway adds the full noise itself, so no other gateway needs to be trusted [47]; the total noise variance in the aggregate grows with the number of clients.

Out of Scope

Active attacks by malicious gateways, namely data poisoning, model poisoning, and Byzantine behaviour, are outside the threat model. Statements in this thesis about privacy do not extend to them.

Roles of Secure Aggregation and Differential Privacy

The two mechanisms address different threats and are not interchangeable (Table 3.3).

Table 3.3: Comparison of protection mechanisms

Aspect	Secure Aggregation (SecAgg$^+$)	Differential Privacy (DP)
Threat addressed	Server inspecting an individual client's update or table	Inference about one data unit from what is released (aggregate, model or synthetic data)
Guarantee	The server recovers only the sum of masked contributions [18, 19]	(ε, δ)-DP [48] with $\delta = 10^{-5}$: the release distribution changes by at most a factor e^{ε}, up to δ, when one unit is added or removed
What it does not do	It does not limit what the aggregate itself reveals, so it receives no differential-privacy credit	It does not hide the individual update from the server unless combined with secure aggregation or applied locally

Privacy Unit

The default unit is the record, that is, one network packet. An ε at record level protects the presence of one packet. For FedDP-Marginal, two larger units are available: the TCP stream (suffix `-strm{m}`) and the capture (suffix `-cap{m}`). Their guarantee holds only under two conditions:

1. each unit is held entirely by one client (the partition is made by unit, not by row);
2. each client uses at most m rows of each unit.

When both hold, a release of T tables has sensitivities $\Delta_1 = m \cdot T$ in ℓ_1 and $\Delta_2 = m\sqrt{T}$ in ℓ_2 (`group_dp.py`), and the noise is calibrated to these values. The CVAE routes keep record-level ε.

Why the Unit Must Be Stated

Packets of one stream, and packets of one capture, are correlated. Under record-level ε, a set of k correlated records from the same source is protected only through group privacy, which

weakens the guarantee to roughly $k\varepsilon$ with a correspondingly larger δ [49]. A statement of ε without its unit therefore says little about whether an attack session as a whole is concealed. The unit statistics of the data are: the training split contains 76,463 TCP streams and 147 capture groups, and a capture contains between 62 and 1,277 rows.

Public Side Information (Not Covered by ε)

The following are treated as known to every party and are outside the accounting:

- the class labels and the number of examples per class;
- client sizes n_i, where they are transmitted in clear (Flower's `num_examples` in the CVAE routes);
- the normalisation statistics, the $\log(1+x)$ choices, the category lists, and the mask of unused categories, computed centrally on the pooled training split before partitioning;
- the hyper-parameter selection, performed on the real validation split.

In the FedDP-Marginal route the client reports `num_examples` as a fixed public constant, so client sizes do not travel through that field (`SPEC_DEVIATIONS.md`, O2.3).

Standard Formulation for the Thesis

We assume an honest-but-curious aggregation server that follows the protocol but may analyse every message it receives. Gateways hold their own traffic and are trusted with it. In the base setting every gateway follows the protocol and adds its share of the noise; we also report the guarantee when only h of the K gateways do so, and an optional threshold t calibrates the noise so that the target ε holds whenever at least t gateways add their share (the others may collude with the server or drop out). In the local-DP variant each gateway adds the full noise, so no other gateway needs to be trusted. Malicious (poisoning or Byzantine) gateways are out of scope.

Secure aggregation hides each gateway's contribution from the server, which learns only the sum; it does not limit what the aggregate reveals and receives no privacy credit. Differential privacy bounds what the released tables, model, or synthetic data reveal about one record (one packet) with $\delta = 10^{-5}$. Class labels, class counts, and client sizes are treated as public side information and are not protected.

3.1.4 Experimental Configurations

Experiment Matrix

Table 3.4 lists the configurations. Every configuration is run with seeds $\{0,1,2\}$. The code of each configuration in the ledger carries the suffixes defined at the end of the table.

Table 3.4: Experiment matrix of PP-FedData

Code	Description	Shared / Released	Protection	Role
B0	IDS trained on pooled real data (TRTR)	None	None	Reference
B1a	B0 with balanced class weights (`class_weight="balanced"`, RF only)	None	None	Reference for class imbalance
B1b	B0 with SMOTE [24] oversampling to class quota	None	None	Reference for classical interpolation
B2	CVAE trained centrally on pooled data	Pooled data	None	Upper generation reference (pools real data)
B3	CVAE with FedAvg, 5 non-IID clients, 30 rounds $\times$ 2 epochs	Model weights	None	Federated baseline ($\varepsilon = \infty$)
M1-eps{1,5,10}	B3 with DP-SGD at each client	Noisy weights	DP (local noise)	DP cost of CVAE route
M2	B3 with Flower SecAgg$^+$	Masked weights	SecAgg$^+$	SecAgg cost of CVAE route
M3-eps5	DP-SGD and SecAgg$^+$ together	Noisy, masked weights	DP and SecAgg$^+$	Combined protection
M1o / M3o-eps{1,5,10}	M1 and M3 tuned per ε at full federated scale	As M1, M3	As M1, M3	Optimisation round O1
MGs-eps{1,5,10}	FedDP-Marginal with Skellam noise, summed via SecAgg$^+$	3 releases of integer count tables	Distributed DP and SecAgg$^+$	Main generator
MGr-eps{1,5,10}	MGs with server post-processing (fusion, IPF [50], smoothing)	As MGs	As MGs	Post-processing (spends no extra ε)
MGd-eps{1,5,10}	Gaussian noise, simulated inside one process	Noisy count tables	Distributed DP (zCDP [51])	Accounting variant
MGl-eps{1,5,10}	Each client adds full noise; no secure aggregation	Noisy count tables	Local DP [47]	No trust in other clients
MGb-eps10	FedDP-Marginal with 2nd-order Bayesian net (PrivBayes-style [9])	Pairwise count tables	Distributed DP (simulated)	Dependency extension
FedMLP / FedMLPcw	MLP classifier trained directly with FedAvg, no generator	Classifier weights	None, DP or SecAgg	Reference (releases no data)

Suffixes: `-strm{m}` protects a whole TCP stream with at most m rows per stream; `-cap{m}` protects a whole capture with at most m rows per capture; `-t{t}` calibrates the noise for t honest clients (default $t = K = 5$; $t \in \{2,3\}$ are additional settings).

Main Configurations of the Framework

Three configurations are designated the framework's main configurations, one per privacy budget: MG-eps1 = MGr-eps1, MG-eps5 = MGs-eps5, MG-eps10 = MGs-eps10, all with $\delta = 10^{-5}$.

Hyper-parameter Selection

Hyper-parameters are tuned on the real validation split with Optuna [52] using the Tree-structured Parzen Estimator (TPE) [53]. The search spaces differ by route (CVAE without DP, CVAE with DP at proxy scale, CVAE with DP at full scale, FedDP-Marginal, IDS side) and are defined in Section 3.3.5.

Robustness and Sensitivity Branches

To assess how far the design conclusions depend on settings fixed in the main study, the matrix includes the branches in Table 3.5. The run status is stated explicitly; branches without a runner are not part of the study.

Table 3.5: Robustness and sensitivity branches

Branch	Change relative to main study	Status
Heterogeneity	Dirichlet $\alpha \in \{0.1, 10.0\}$ (main $\alpha = 0.5$)	Run for MGs (one seed per setting)
Federation size	$K \in \{10, 20\}$ (main $K = 5$)	Run for MGs (one seed) and for MGl (three seeds)
Label resolution	11-class mode separating DoS and DDoS	Run through robustness module for reference and FedDP-Marginal
A4	`split_seed` changed so other capture groups form validation/test	Run with two alternative choices
A5	At most 20 packets per TCP stream in training pool (`max_rows_per_stream` = 20)	Run
A2, A3	Class-imbalance ratios of training pool; 11-class matrix	Declared as `needs_data` without a runner; not run

Configurations Written but Not Run

Two components exist as code and tests but were not executed after the change of primary generator (Section 3.1.1): M3f, a distributed DP-FedSGD variant of the CVAE, and a sweep of the SecAgg^{+} quantisation width for the CVAE weights. Both are listed again among the limitations (Section 3.4).

3.1.5 Requirement-Driven Recommendation Rule

Motivation and Relation to Rule R6

The original specification fixed a single rule, R6: among the configurations whose membership-inference AUC is at most 0.55, whose ε (maximum over clients and worst seed) is at most 5 when DP is used, and whose time per round is at most three times that of B3, choose the one with the highest TAug macro-F1 of the Random Forest. Ties within the seed noise are broken by the strength of the formal protection. The thresholds are heuristics fixed by the research team (Section 3.4). R6 is retained as a reference rule. A single fixed criterion cannot express deployments that differ in trust, bandwidth, and privacy unit, so the framework adds a requirement-driven rule, implemented as `ppfeddata recommend` (`recommend.py`), which takes a deployment requirement as input.

Requirement Formulation

A requirement is the tuple:

$$\begin{aligned}\mathscr{R} = \langle &\mathrm{Trust}_{\mathrm{server}}, \mathrm{Trust}_{\mathrm{clients}}, \varepsilon_{\max}, \mathrm{BW}_{\mathrm{round}}, \mathrm{BW}_{\mathrm{total}}, \\ &\mathrm{Rounds}_{\max}, \mathrm{Unit}, \mathrm{Priority}, \mathrm{Recall}_{\mathrm{rare,min}}, \mathrm{MIA}_{\max}, h\rangle\end{aligned} \tag{3.1}$$

where $\mathrm{Trust}_{\mathrm{server}}$ and $\mathrm{Trust}_{\mathrm{clients}}$ state whether the server and the other clients are trusted; $\varepsilon_{\max}$ is the record-level (or unit-level) budget, or none if no DP is required; $\mathrm{BW}_{\mathrm{round}}$, $\mathrm{BW}_{\mathrm{total}}$, and $\mathrm{Rounds}_{\max}$ bound the gateway bandwidth per round, over the whole run, and in rounds; $\mathrm{Unit} \in \{\mathrm{packet}, \mathrm{stream}, \mathrm{capture}\}$ is the unit ε must cover; $\mathrm{Priority} \in \{\mathrm{macro}, \mathrm{binary}, \mathrm{rare}\}$ selects TSTR macro-F1, binary F1, or mean rare-class recall; $\mathrm{Recall}_{\mathrm{rare,min}}$ is a floor on the rare-class recall; $\mathrm{MIA}_{\max}$ is the maximum acceptable membership-inference AUC; and h is the number of clients assumed to add their noise share.

Procedure

Inputs are the rows of the scorecard, which are produced from the run ledger; no figure is typed by hand. The rule applies five steps:

1. **Validity filter:** A row is considered only if its reported quantities correspond to what its name claims. For example, a DP configuration with a component computed from data outside the ε accounting (residual noise computed from real training data) is not valid; a directly federated classifier is a reference and is excluded from the generator front.

2. **Trust filter:** If the server is not trusted, only rows with secure aggregation or local DP remain. If clients may collude, the ε used for a distributed-DP row is the one that holds with the assumed number of honest clients (one honest client by default, or h when it is specified); local-DP rows keep their ε because each client adds the full noise.

3. **Constraint filter:** Rows that exceed the ε budget (with a 2% tolerance), the per-round or total bandwidth, the number of rounds, or the MIA bound, that do not protect the requested unit (a row protecting a packet does not satisfy a stream or capture requirement), or whose rare-class recall is below the floor, are removed. Missing cost figures do not exclude a row; they are reported as not available.

4. **Pareto front:** Among the remaining rows, configuration A dominates B if A is at least as good on every axis and better on at least one. The axes are the three utility measures (TSTR macro-F1, binary F1, rare-class recall), ε under the assumed trust, the presence of secure aggregation, MB per round, and total MB. A utility difference counts only if it exceeds the seed standard deviation of both configurations; ε values within 2% and cost values within 5% are treated as equal; a configuration with secure aggregation is preferred to one without when the other axes are equal.

5. **Selection and ties:** The configuration with the best value on the requested priority is selected, and every configuration within the seed standard deviation of it is listed as tied.

Deployment Scenarios

Table 3.6 defines ten illustrative requirements. Only the inputs are specified here; the configuration chosen for each is a result and is reported in Chapter 4.

Table 3.6: Deployment scenarios (inputs only)

Scenario	Setting	Requirement
`lab`	One operator owns server and all gateways	Server and clients trusted; no DP required; priority macro
`cloud-aggregator`	Gateways of one company, 3rd-party cloud	Server not trusted; clients trusted; $\varepsilon \leq 5$; priority macro
`consortium`	Brokers of different companies may collude	Server not trusted; clients may collude; $\varepsilon \leq 10$; priority macro
`strict-privacy`	Sensitive deployments (e.g., healthcare IoT)	Server not trusted; clients trusted; $\varepsilon \leq 1$; priority macro
`low-bandwidth`	NB-IoT, LoRa or metered cellular	Server not trusted; $\varepsilon \leq 10$; ≤ 2 MB/round; $\leq$ 100 MB total; priority macro
`rare-attacks`	Rare attacks (DELAYED, SYN, INVALID, WILL)	Server not trusted; clients trusted; $\varepsilon \leq 5$; priority rare
`honest-majority`	Five operators, at most 2 may collude	Server not trusted; ≥ 3 honest clients; $\varepsilon \leq 10$; MIA AUC < 0.55; priority macro
`gateway-dropout`	Up to 2 of 5 gateways drop out / leak noise	Server not trusted; ≥ 3 honest clients; $\varepsilon \leq 5$; priority macro
`session-privacy`	ε must cover a whole TCP session	Server not trusted; $\varepsilon \leq 5$; unit = stream; priority macro
`capture-privacy`	ε must cover a whole capture	Server not trusted; $\varepsilon \leq 10$; unit = capture; priority macro

3.2 Data Collection Methods

3.2.1 Dataset

The data source is the DoS/DDoS-MQTT-IoT dataset of Alatram et al. [2], obtained from the authors' official Kaggle release [54]. It contains normal IoT traffic and several denial-of-service (DoS) and distributed denial-of-service (DDoS) attacks that abuse the MQTT protocol [55]. The release is distributed in several formats; this study uses a CSV form of 59,624,826 rows (about 13 GB).

Each row is one network packet described by 33 feature columns. The Normal traffic has two additional columns, `Label` and `Capture_ID`; `Capture_ID` assigns the `group_id` used for splitting, whereas attack rows take their `group_id` from the name of the source file. An inventory step confirms that the row counts match the expected totals. Table 3.7 gives the composition by class.

Table 3.7: Composition of the DoS/DDoS-MQTT-IoT dataset

Class	**DoS**	**DDoS**	**Total rows**	**Share**
NORMAL	—	—	45,580,102	76.44%
BCF (Basic Connect Flooding)	5,825,026	6,455,848	12,280,874	20.60%
DELAYED (Delayed Connect Flooding)	291,988	309,554	601,542	1.01%
SYN (SYN TCP Flooding)	103,828	497,611	601,439	1.01%
INVALID (Invalid Subscription Flooding)	109,150	181,190	290,340	0.49%
WILL (Connect Flooding with WILL payload)	114,474	156,055	270,529	0.45%
Total			**59,624,826**	**100.00%**

Data Provenance and Verification

The attack traffic is used as published: the 48 attack CSV files exported by the dataset authors [2], whose paths and sizes match the Kaggle release [54]. The release provides the Normal traffic only as 49 PCAP files, so every Normal packet was exported with TShark 4.4.7 [56] into the same 33 columns, using a script kept in the project repository.

The export is complete (the number of rows equals the number of packets in all 49 captures, 45,580,102 rows) and reproducible (re-extracting three captures yields byte-identical files). To validate the procedure, it was applied to the attack PCAP files, for which the authors' CSV files serve as ground truth: 23 of the 25 features used by the models match the authors' values in at least 99.98% of the rows. The two remaining differences are explained by the authors' export settings, namely a display filter that keeps only the packets sent to the broker and two-pass dissection; adding both settings reproduces the authors' rows in all 48 captures. The Normal export therefore contains every packet that the authors' settings would select from the same PCAP files, plus the packets sent by the broker; the consequences of this difference are discussed in Section 3.4. Table 3.8 summarises the checks.

Table 3.8: Data provenance verification

Question	Method	Result
Are the files those of the release?	File paths and sizes compared with the public Kaggle manifest [54]	411/411 files match (49 attack CSVs, 313 attack PCAPs, 49 Normal PCAPs)
Does the Normal CSV contain all packets?	Row counts compared with the packet counts of `capinfos` [56]; frame numbers checked to run $1, \ldots, N$	Complete match for 49/49 captures (45,580,102 rows)
Is the extraction reproducible?	Three captures re-extracted and SHA-256 digests compared	Files are byte-identical
Does the procedure give correct feature values?	Script applied to the 48 attack PCAPs; output compared row by row with the authors' CSVs	23/25 model features match in $\geq$ 99.98% of rows
What explains the remaining differences?	Traced to the authors' two-pass dissection and display filter	With both settings the authors' rows are reproduced; the remaining 0.04% stem from the Wireshark version

3.2.2 Label Harmonisation and Rare Classes

The labels are harmonised into a six-class mode: NORMAL, BCF, DELAYED, SYN, INVALID, and WILL. In this mode, the DoS and DDoS variants of the same attack family are merged into one class. The merge is supported by leakage check C5 (Section 3.2.6), which shows that DoS and DDoS variants of the same family are practically indistinguishable at packet level. The original eleven-class mode remains available in the pipeline but is used only for the robustness branch (Table 3.5).

In this study, the rare classes are the attack classes whose training quota is at most 5,000 rows, namely DELAYED, SYN, INVALID, and WILL. Their rarity in training is a deliberate consequence of the quota design rather than a lack of raw data: as Table 3.7 shows, the raw dataset contains between 270,529 and 601,542 packets for each of them.

The quotas create an imbalance of 60:1 between the majority class (NORMAL, 60,000 rows) and the smallest class (WILL, 1,000 rows), which makes the evaluation of the generators demanding [3]. The validation and test splits, in contrast, are class-balanced (2,000 and 5,000 rows per class respectively), so that the evaluation measures are not dominated by the majority

class.

3.2.3 Schema Harmonisation and Feature Selection

Preprocessing starts by harmonising the schema of the raw packet exports. Column names are converted to snake case. Wireshark text labels (such as `message_type`, `qos_level_*`, and `requested_qos`) are normalised to canonical codes, and boolean indicators (True/False, Set/Not set) are mapped to 0/1. Redundant columns with the suffix `_1` are removed. For multi-valued cells that contain several MQTT messages, only the first element is kept, in line with the Normal export, which used the TShark option `occurrence=f`; the `n_mqtt_msgs` column is therefore dropped. Rows are then restricted to those whose protocol is TCP or MQTT, followed by a format audit of the time-based tokens.

To prevent leakage [57, 58], identifier and metadata columns are removed:

- `no` (row sequence), `epoch_time` (absolute timestamp), and `time_since_reference_or_first_frame` (which reveals the start time of the capture);
- `source`, because sender IP addresses would let a model memorise machine identities instead of learning traffic patterns;
- the free-text payload in `info`, and the `label` and `capture_id` columns, which exist only in the Normal traffic.

`stream_index` is kept only as grouping metadata and is not an input feature. The raw dataset contains no MAC address or port columns. After the feature selection decisions, the final schema has $D = 60$ input columns (Table 3.9).

3.2.4 Group-Based Split and Quota Sampling

The eleven sub-classes (NORMAL and the DoS and DDoS variants of the five attack families) are split by group: whole capture groups are assigned to the training, validation, or test split according to a fixed `split_seed`. This keeps highly correlated packets of the same TCP stream or capture session from appearing in more than one split [58].

The source data limits this approach: 8 of the 11 sub-classes come from a single capture file. For these sub-classes, the validation and test rows are taken as consecutive blocks of the same capture used for training (block split), separated by buffers of 1,000 rows to reduce

Table 3.9: Final feature schema ($D = 60$)

Feature type	Columns	Dim.
Numeric	`time_delta_from_previous_displayed_frame`, `frame_length_on_wire_2`, `irtt`, `time_since_first_frame_in_this_tcp_stream`, `tcp_segment_len`, `calculated_window_size`, `keep_alive`, `user_name_length`, `password_length`, `will_message_length`, `will_topic_length`, `topic_length`, `msg_len`	13
Binary	`syn`, `reset`, `acknowledgment`, `retain`, `clean_session_flag_2`, `will_retain`, `will_flag`	7
Missingness flags	12 `_is_na` indicator columns	12
Categorical (one-hot)	`message_type` (12), `qos_level_1` (5), `qos_level_2` (4), `requested_qos` (3), `protocol` (4)	28
Total		**60**

adjacency leakage. TCP streams that straddle a block boundary are dropped entirely. This removes between 1.2% and 9.9% of the eligible rows of each affected sub-class and biases the sample slightly toward shorter connections.

After the split, rows are drawn at random without replacement according to the class quotas in Table 3.10. The training quotas impose a strong imbalance, whereas the validation and test quotas are balanced.

Table 3.10: Class quotas of the training, validation, and test splits

Class	Train	Validation	Test
NORMAL	60,000	2,000	5,000
BCF	20,000	2,000	5,000
DELAYED	3,000	2,000	5,000
SYN	3,000	2,000	5,000
INVALID	1,500	2,000	5,000
WILL	1,000	2,000	5,000
Total	**88,500**	**12,000**	**30,000**

The 30,000-row test split is locked and used only for the final report. All hyper-parameter tuning, design choices, and threshold calibration use the validation split only.

3.2.5 Preprocessing and Encoding

The preprocessing pipeline is fitted on the training split only; the validation and test splits are transformed with the stored parameters. Missing or non-applicable numeric values are imputed with 0, and an indicator column `_is_na` is added for every feature whose missing rate in the training split lies strictly between 0.005 and 0.995.

Numeric features with a skewness greater than 2 are transformed with $\log(1+x)$; time-delta features are first multiplied by 1,000 (seconds to milliseconds). Numeric columns are then standardised to zero mean and unit variance and clipped at $\pm 5\sigma$ to limit the influence of extreme values. Categorical features are one-hot encoded. To bound the dimension, only the ten most frequent training tokens of each categorical column are kept; other valid values are grouped into an `OTHER` token and missing values are mapped to a `NONE` token.

The normalisation statistics, category lists, and imputation parameters are fitted centrally on the pooled training split. This is a limitation of the method: it assumes a centralised view of the data before federation, and these statistics lie outside the privacy budget ε (Section 3.1.3).

3.2.6 Leakage and Label-Reliability Checks

Before any synthetic data is generated, a set of diagnostic checks verifies that the learning task is not made artificially easy by the schema and that the attack labels are reliable. Each check uses a Random Forest [41] with 100 trees, evaluated on the validation split with seeds $\{0,1,2\}$.

The suite consists of five checks (C1–C5). Each has a heuristic threshold that flags an anomaly for manual inspection or a change of schema (Table 3.11).

Table 3.11: Leakage and label-reliability checks

Code	Check	Flag threshold
C1	Presence of each column across classes, to find attributes structurally bound to particular attacks	A column is empty in one class but populated in another
C2	Random Forest trained only on the `_is_na` missingness indicators	Macro-F1 > 0.5
C3	Class-balanced Random Forest trained on single raw columns, and ablations that omit given features	Macro-F1 or best per-class F1 > 0.9 on any single feature
C4	Gap between a random row split and the group-based stream split	Difference (random minus group) > 0.05
C5	Binary task separating the DoS and DDoS variants of the same attack family	F1 < 0.6

3.3 Sampling Data Analysis Techniques

3.3.1 Experimental Environment and Reproducibility

Environment

All experiments ran on one Windows 11 machine (build 10.0.26200) with an Intel x86-64 CPU and 16.9 GB of RAM, using the CPU only. Federated learning is simulated with Flower and Ray; each client is a Ray actor sharing the machine's CPU, so no network latency is measured (see Section 3.4). The values come from the environment record `results/repro/environment.json` and the project's deviation log (Table 3.12).

Random Seeds

Every experiment is run with seeds $\{0, 1, 2\}$. The seed controls weight initialisation, the client partition, and the sampling of synthetic data. `utils.set_seed` seeds Python's `random`, NumPy, and PyTorch; the federated and DP code additionally derives per-client and per-round generators from the seed.

Table 3.12: Experimental environment specification

Component	Value
Operating system	Windows 11, build 10.0.26200 (AMD64)
Processor	Intel64 Family 6 Model 186 Stepping 2 (GenuineIntel); CPU only, 10 threads in benchmark
Memory	16.9 GB
Language	Python 3.13.5
Federated simulation	Flower 1.39.0 [43]; Ray 2.59.0
Deep learning and DP	PyTorch 2.14.1; Opacus 1.6.0 [45]
Machine learning and data	scikit-learn 1.9.1; imbalanced-learn 0.14.2; NumPy 2.3.2; pandas 2.3.1; SciPy 1.18.1; PyArrow 25.0.1
Hyper-parameter search	Optuna 5.0.0 [52] (TPE sampler [53])
Other	Joblib 1.6.0; Matplotlib 3.10.5; PyYAML 6.0.3; Streamlit 1.65.0; psutil 7.2.2

Measures Supporting Reliability and Reproducibility

- **Central run ledger:** Each run appends one row to `results/runs.csv` with a configuration hash and the Git commit of the code that produced it. The environment record of the reference run shows commit `9ee2a85e` and configuration hash `8c86c0b143d4`.
- **Checkpoint and resume:** Long training jobs save state; re-running a command compares against the ledger and skips completed configurations.
- **Automated reporting:** The tables, intervals, and figures of the final report are generated by code (`ppfeddata aggregate`, `ppfeddata interpret`); no result is typed by hand. A check recomputes macro-F1 from the saved prediction files and compares it with the ledger.
- **Reproducibility bundle:** `results/repro/` holds the exact package list (`pip_freeze.txt`, identical to `requirements-lock.txt`), the configuration files, the environment record, and a manifest with SHA-256 digests of the ledger, the split and schema manifests, and the report.
- **Acceptance command:** `python -m ppfeddata.cli accept` checks the repository

against the Definition of Done and writes a checklist of twelve items (D1–D10, with D6 split into three). This command does not run the test suite; the suite is run separately.

- **Clean-copy re-execution:** Phases 1–5 (inventory, harmonisation, sampling, preprocessing, leakage check) were re-run from the raw data in a clean copy of the repository with a fresh virtual environment, and the processed train, validation, and test arrays were compared with those of the main checkout. Phases 6–14 were not re-run in the clean copy.
- **Controlled trial:** A trial run of all ten configurations of the base matrix with seed 0 was compared row by row with the ledger of the main checkout (`SPEC_DEVIATIONS.md`, 11.6).
- **Leakage checks:** The data pipeline is guarded by five checks (C1–C5, Section 3.2.6), and the real test split is verified against a stored hash.

3.4 Limitations of the Methodology

The limitations below arise from the design of the study and the scope of the techniques. The full list is maintained in `limitations.py`, which feeds the final report and the README; this section groups them. None of the statements below reports a result.

Data and Features

- Only one dataset is used; the study offers no evidence about other traffic or networks.
- The Normal traffic was re-exported from the original PCAP files and also contains the packets sent by the broker, which the authors' export settings exclude (Section 3.2.1).
- Records are single packets, and the generators produce packets independently. Session-level temporal structure is therefore not reproduced, and the time-based features depend on how the captures were exported.
- Eight of the eleven sub-classes originate from a single capture file, so their validation and test rows are consecutive blocks of the same capture and not other captures. Removing TCP streams that straddle two blocks (`drop_cross_block_streams`) reduces leakage

between splits but biases the sample toward short connections; the share removed per sub-class is documented in the split manifest.

- The pipeline keeps only rows with protocol TCP or MQTT, keeps the first element of multi-valued cells, clips the tail of the time-gap feature at 5σ, and treats nearly empty columns with the statistics of the rows where they apply plus an indicator.
- DoS and DDoS are merged in the six-class mode used for the experiment matrix.
- The validation and test splits are class-balanced (5,000 test rows per class) and do not follow the class proportions of real traffic.

Privacy Scope

- ε is stated at record level (one packet) by default. Stream- and capture-level protection is optional for FedDP-Marginal and requires each unit to be held by one client and capped at m rows. Under record-level ε, an attack session or capture is protected only through the weaker group bound [49].
- Class labels, class counts, and client sizes are not protected (Section 3.1.3).
- Normalisation statistics are computed centrally on the pooled training split, as if each client had shared its data once. A real federation would have to compute them securely or fix them beforehand.
- Hyper-parameter selection on the real validation split is not private and lies outside ε.
- Distributed DP assumes that enough clients add their noise share; the guarantee degrades as the number of honest clients falls, which is why ε is reported as a function of h.
- Accounting differs between variants: the Skellam variant uses the RDP bound of Agarwal et al. [22] and the RDP-to-(ε, δ) conversion of Opacus [45], while the Gaussian variant uses zCDP [51] with a looser conversion, so part of the difference between the two at the same ε comes from the conversion and not from the mechanism.
- Malicious gateways (poisoning, Byzantine behaviour) are outside the threat model.

Generators

- FedDP-Marginal represents dependencies with a Chow–Liu tree [10], so each attribute depends on one parent; a second-order structure (MGb) [9] is only an option.
- Only one CVAE architecture is studied, and its hyper-parameter search used one seed per trial.

Evaluation and Statistics

- Three seeds are used for every configuration. Federated training can be sensitive to very small numerical perturbations, so a standard deviation over three seeds is a rough estimate.
- The bootstrap intervals and decision rules reflect the sampling of test rows (or whole streams) and of training seeds; they do not cover the choice of capture groups, the hyper-parameters, or the data sampling. Sensitivity A4 explores only two alternative choices of validation and test groups.
- The decision thresholds (for example the membership-inference AUC bound of 0.55, the ε bound of 5, and the overhead ratio of 3) are heuristics fixed by the research team and not derived from data.
- Each membership-inference family tests one attack at record level and does not test group-level inference; each attack must pass a positive control to be informative.
- The decision rule does not use p-values or a multiple-comparison correction.

Execution Platform

- Federated learning is simulated on one CPU machine with Ray actors; no network latency, packet loss, or constrained-device behaviour is measured, and the reported times per round are compute times on this machine.

Baselines Not Compared and Parts Not Run

- **Not compared:** FLAIM and AIM (the Private-PGM dependency was not installed), DP-CTGAN or other deep generators in the federation, federated SMOTE, and a directly trained federated classifier with distributed DP.
- **Written but not run after the change of primary generator:** M3f (distributed DP-FedSGD for the CVAE) and the SecAgg$^+$ quantisation-width sweep for the CVAE. Robustness branches A2 and A3 have configuration files but no runner.

Chapter 4

Experimental and Results

Provide a concise and precise description of the experimental results, their interpretation as well as the experimental conclusions that can be drawn. You should discuss the results and how they can be interpreted in perspective of previous studies and of the working hypotheses.

4.1 Introduction

4.2 Presentation of Data

4.3 Analysis of Results

4.4 Interpretation of Results

4.5 Comparison with Literature

4.6 Implications of the Results

Chapter 5

Discussion

The findings and their implications should be discussed in the broadest context possible and limitations of the work highlighted.

5.1 Restate the Research Problem or Objectives

5.2 Summarize Key Findings

Chapter 6

Conclusion and Future Work

The 'conclusions' are a key component of the paper. It should complement the 'abstract' and is normally used by experts to value the paper's engineering content. A conclusion is not merely a summary of the main topics covered or a re-statement of your research problem, but a synthesis of key points and, if applicable, where you recommend new areas for future research.

6.1 Conclusion

6.2 Future Work

Appendix A

If Any

References

[1] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Computer Networks*, vol. 54, no. 15, pp. 2787–2805, 2010.

[2] A. Alatram, L. F. Sikos, M. Johnstone, P. Szewczyk, and J. J. Kang, "DoS/DDoS-MQTT-IoT: A dataset for evaluating intrusions in IoT networks using the MQTT protocol," *Computer Networks*, vol. 231, p. 109809, 2023.

[3] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR, 2017, pp. 1273–1282.

[5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[6] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, "Federated learning for resource-constrained IoT devices: Panoramas and state-of-the-art," *Machine Learning and Knowledge Extraction*, 2022. [Online]. Available: https://arxiv.org/abs/2002.10610

[7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/60a6c4002cc7b29142def8871531281a-Paper.pdf

[8] M. R. Behera, S. Upadhyay, S. Shetty, S. Priyadarshini, P. Patel, and K. F. Lee, "FedSyn: Synthetic data generation using federated learning," *arXiv preprint arXiv:2203.05931*, 2022. [Online]. Available: https://arxiv.org/abs/2203.05931

[9] J. Zhang, G. Cormode, C. M. Procopiuc, D. Srivastava, and X. Xiao, "PrivBayes: Private data release via Bayesian networks," *ACM Transactions on Database Systems*, vol. 42, no. 4, pp. 25:1–25:41, 2017.

[10] C. K. Chow and C. N. Liu, "Approximating discrete probability distributions with dependence trees," *IEEE Transactions on Information Theory*, vol. 14, no. 3, pp. 462–467, 1968.

[11] R. McKenna, D. Sheldon, and G. Miklau, "Graphical-model based estimation and inference for differential privacy," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97. PMLR, 2019, pp. 4435–4444. [Online]. Available: https://arxiv.org/abs/1901.09136

[12] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau, "AIM: An adaptive and iterative mechanism for differentially private synthetic data," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2599–2612, 2022.

[13] A. Vyas, S. Balamurugan, P. Kumar, and K. Kotecha, "Privacy-preserving federated learning for intrusion detection in IoT environments: A survey," *IEEE Access*, vol. 12, pp. 130 452–130 478, 2024.

[14] R. Manzano, M. Zaman, D. Upadhyay, N. Goel, and S. Sampalli, "Federated learning framework based on distributed storage and diffusion model for intrusion detection on IoT networks," *IEEE Access*, vol. 13, pp. 79 571–79 595, 2025.

[15] J. Nakayiza and J. H. Park, "PureFL: Trust-weighted federated learning with noise-resilient homomorphic encryption for blockchain-based IoV networks," *IEEE Internet of Things Journal*, 2026.

[16] P. Ruzafa-Alcázar, P. Fernández-Maimó, F. Gómez Mármol, and G. Martínez Pérez, "Intrusion detection based on privacy-preserving federated learning for the industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 2, pp. 1145–1154, 2023.

[17] C. Bessaa, M. R. Senouci, and S. Bouzefrane, "FL-SQMPC: Secure quantized aggregation for mitigating gradient inversion attacks in federated IoT intrusion detection," *IEEE Journal on Selected Areas in Sensors*, 2026.

[18] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, and M. Raykova, "Secure single-server aggregation with (poly)logarithmic overhead," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. ACM, 2020, pp. 1253–1269.

[19] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. ACM, 2017, pp. 1175–1191.

[20] C. Gutti, R. Kumar, and S. Sharma, "Federated learning for distributed IoT security: A privacy-preserving approach to intrusion detection," *IEEE Access*, vol. 13, pp. 25 410–25 425, 2025.

[21] P. Kairouz, Z. Liu, and T. Steinke, "The distributed discrete Gaussian mechanism for federated learning with secure aggregation," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139. PMLR, 2021, pp. 5201–5212. [Online]. Available: https://arxiv.org/abs/2102.06387

[22] N. Agarwal, P. Kairouz, and Z. Liu, "The Skellam mechanism for differentially private federated learning," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 34. Curran Associates, Inc., 2021, pp. 5037–5049. [Online]. Available: https://arxiv.org/abs/2110.04995

[23] Z. Deng, A. Torim, S. Ben Yahia, and H. Bahsi, "Generative AI in intrusion detection systems for internet of things: A systematic literature review," *IEEE Open Journal of the Communications Society*, vol. 6, pp. 4689–4717, 2025.

[24] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[25] J. Yang, V. Govindarajan, L. Y. Por, Z. A. Shaikh, Q. Xin, P. Bhattacharya, A. A. Khan, and Y. Wang, "DDoS attack detection in consumer IoT-based healthcare systems using improved off-policy proximal policy optimization and generative adversarial network," *IEEE Transactions on Consumer Electronics*, vol. 71, no. 4, pp. 11 662–11 675, Nov. 2025.

[26] S. L. Jacob and H. P. Sultana, "GBOA_SR-ShuffleNet: An explainable federated learning framework for privacy-preserving intrusion detection in IoT," *Scientific Reports*, vol. 16, p. 5621, 2026.

[27] R. Anandhi and G. U. Srikanth, "Federated sparse-learning framework for detecting low-rate stealth attacks in resource-constrained IoT devices," *Scientific Reports*, vol. 16, p. 21298, 2026.

[28] S. Durga, M. G. Shankar, E. Daniel, and B. G. Varghese R., "Enhancing security in IoMT using federated TinyGAN for lightweight and accurate malware detection," *Scientific Reports*, vol. 16, p. 7116, 2026.

[29] H. N. Cunha Neto, J. Hribar, I. Dusparic, D. M. F. Mattos, and N. C. Fernandes, "A survey on securing federated learning: Analysis of applications, attacks, challenges, and trends," *IEEE Access*, vol. 11, pp. 41 928–41 953, 2023.

[30] W. Yao, H. Zhao, and H. Shi, "Privacy-preserving collaborative intrusion detection in edge of Internet of Things: A robust and efficient deep generative learning approach," *IEEE Internet of Things Journal*, vol. 11, no. 9, pp. 15 704–15 722, 2024.

[31] X. He, Q. Chen, L. Tang, W. Wang, and T. Liu, "CGAN-based collaborative intrusion detection for UAV networks: A blockchain-empowered distributed federated learning approach," *IEEE Internet of Things Journal*, vol. 10, no. 1, pp. 120–132, 2023.

[32] S. Salim, N. Moustafa, and A. Almorjan, "Responsible deep-federated-learning-based threat detection for satellite communications," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 4807–4819, 2025.

[33] S. Jiao, J. Wang, Z. Tong, Z. Wang, L. Tan, and X. Zhang, "Foundation-model-based federated learning for intrusion detection in drone-aided industrial IoT," *IEEE Internet of Things Journal*, vol. 12, no. 22, pp. 46 889–46 901, 2025.

[34] R. Shettar, R. Coelho, and S. S. Veerasagar, "SYNAPSE: Synthetic network-traffic agent pipeline for federated edge security," *IEEE Access*, vol. 14, pp. 116 318–116 335, 2026.

[35] D. Hamouda, M. A. Ferrag, N. Benhamida, H. Seridi, and M. C. Ghanem, "Revolutionizing intrusion detection in industrial IoT with distributed learning and deep generative techniques," *Internet of Things*, vol. 26, p. 101149, 2024.

[36] S. Maddock, G. Cormode, and C. Maple, "FLAIM: AIM-based synthetic data generation in the federated setting," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. ACM, 2024, pp. 2165–2176.

[37] S. Pentyala, M. Pereira, and M. De Cock, "CaPS: Collaborative and private synthetic data generation from distributed sources," in *Proceedings of the 41st International Conference on Machine Learning (ICML '24)*, ser. Proceedings of Machine Learning Research, vol. 279. PMLR, 2024, pp. 173–198.

[38] X. Li, F. Gao, X. Wei, T. Wang, C. Shen, and J. Yang, "HeteroFedSyn: Differentially private tabular data synthesis for heterogeneous federated settings," *arXiv preprint arXiv:2603.08832*, 2026.

[39] S. Duan, C. Liu, P. Han, X. Jin, X. Zhang, T. He, H. Pan, and X. Xiang, "HT-Fed-GAN: Federated generative model for decentralized tabular data synthesis," *Entropy*, vol. 25, no. 1, p. 88, 2022.

[40] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.

[41] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[42] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015, pp. 3483–3491.

[43] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[44] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zheng, "Deep learning with differential privacy," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. ACM, 2016, pp. 308–318.

[45] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Tian, N. Christofides, R. Gu, M. Ling, H. Yu, B. Wang, Z. Gu *et al.*, "Opacus: User-friendly differential privacy library in PyTorch," *arXiv preprint arXiv:2109.12298*, 2021.

[46] O. Goldreich, *Foundations of Cryptography: Volume 2, Basic Applications*. Cambridge, U.K.: Cambridge University Press, 2004.

[47] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" *SIAM Journal on Computing*, vol. 40, no. 3, pp. 793–826, 2011.

[48] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Theory of Cryptography Conference (TCC)*. Springer, 2006, pp. 265–284.

[49] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.

[50] W. E. Deming and F. F. Stephan, "On a least squares adjustment of a sampled frequency table when the expected marginal totals are known," *The Annals of Mathematical Statistics*, vol. 11, no. 4, pp. 427–444, 1940.

[51] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," in *Theory of Cryptography Conference (TCC)*. Springer, 2016, pp. 635–658.

[52] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*. ACM, 2019, pp. 2623–2631.

[53] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011, pp. 2546–2554.

[54] A. Alatram, L. F. Sikos, M. Johnstone, P. Szewczyk, and J. J. Kang, "DoS/DDoS-MQTT-IoT dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/alaaalatram/dosddos-mqtt-iot

[55] A. Banks and R. Gupta, "MQTT version 3.1.1," OASIS Standard, Oct. 2014. [Online]. Available: http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[56] Wireshark Foundation, "Wireshark: Network protocol analyzer (TShark and capinfos)," Version 4.4.7, 2025. [Online]. Available: https://www.wireshark.org

[57] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[58] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proceedings of the 31st USENIX Security Symposium*. USENIX Association, 2022, pp. 3971–3988.